

Python Modules

What is a Module?

A module in python is a file(.py) that contains python code(like functions, variables, or classes)which can be reuse in another program.

Real-Life Example:

Think of a module like a book:

- Each book = one module
- Chapters = functions/classes
- You can open any book when needed

Why Modules are Important:

- Code reuse(write once, use many times)
- Better organization
- Easy debugging
- Reduces code length

Create a Module :

To create a module: write python code and save it with **filename.py** “.py” extension.

Example :

```
def exa_1(name):  
    print("Hello", name)  
exa_1("priya")
```

- Save this code in a file named mymodule.py

Use a Module :

Now we can use the module we just created, by using the “**import**” statement.

Import : it is a keyword used to bring code from one module into another program.

Example :

Import the module named mymodule, and call the greeting function:

```
import lm  
lm.exa_1("priya")
```

- If there is a file named `lm`, and now we are importing it into a new file, the reason is that the file `lm` contains code. In the new file, we are calling a function from that file. For example, `exa_1()` is the function name from the previous file.

Note: when using a function from a module, use the syntax:

`modulename.functionname.`

Variables in Module:

The Module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc):

Example:

Save this code in the file `lm.py`

```
car1={
    "model":2001,
    "company":"audi",
    "price" : 10.20,
    "color":"Blue"
}
```

- Import the module named `lm.py`, access the `car1` dictionary:

```
import lm
a=lm.car1["color"]
print(a)
```

output : Blue

Naming a Module:

You can change the module file whatever you like, but it must have the file extension `.py`

Renaming a Module:

You can create an alias when you import a module, by using the “`as`” keyword:

Example:

Create an alias for `lm` called `mx`:

```
import lm as mx
a=mx.car1["color"]
print(a)
```

output : Blue

Built – in Modules :

Python already provides modules.

Examples:

- math → Calculations
- platform → System info
- random → random values
- datetime → date and time
- sys → specific operations
- os → system information
- time → related function
- calendar → used to work with calendars
- statistics → statistical calculations
- string → constants and utility functions
- re(regular expressions) → pattern matching
- json → work with json data
- collections → special data structures
- itertools → looping operations
- functools → higher-order functions

1.math: A module used to perform mathematical operations like square root, power, and rounding.

Functions : sqrt(), pow(), ceil(), floor(),max(),min(),abs()

- The min() and max() functions can be used to find the lowest or highest value in an iterable:

Example:

```
x = min(5,10,25)
y = max(5,10,25)
print("minimum value:",x)
print("maximum value:",y)
```

output: minimum value: 5

maximum value: 25

- The `abs()` function returns the absolute (positive) value of the specified number:

Example:

```
x = abs(-7.25)
print(x)
```

output: 7.25

- The `pow(x,y)` function returns the value of x to the power of y(x^y)

Example :

Return the value of 4 to the power of 3(same like $4*4*4$)

```
x = pow(4,3)
print(x)
```

output: 64

Code Challenge : Math

Test your understanding of Python math by completing a small coding challenge.

Instructions

Inside the editor, complete the following steps:

1. Print the lowest value of **5** and **10** using `min()`
2. Print the highest value of **5** and **10** using `max()`
3. Print the absolute value of **-7.25** using `abs()`
4. Print the value of **4** to the power of **3** using `pow()`.

```
x=min(5,10)
print("minimum values:",x)
y=max(5,10)
print("maximum value:",y)
z=abs(-7.25)
print(z)
i=pow(4,3)
print(i)
```

output: minimum values: 5

maximum value: 10

7.25

64

Try Except

Exception Handling : Exception handling is a process in python used to handle runtime errors so that the program does not stop and can continue the execution process.

These exceptions can be handled using the “try” statement:

- try : try block is used to test a block of code for errors.

Example :

```
try:  
    print (x)
```

output: SyntaxError: expected 'except' or 'finally' block

Note: a try block cannot be used alone; it must be followed by an except or finally block.

- except : the except block is used to handle the error raised in the try block.

Example :

```
try:  
    print (x)  
except:  
    print("invalid input")
```

output: invalid input

- finally : the finally block is used to execute code whether an error occurs or not.

Example :

```
try:  
    print (x)  
except:  
    print("invalid input")  
finally:  
    print("completed")
```

output : invalid input

completed

Many Exceptions

You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error:

Example :

Print one message if the try block raises a `NameError` and another for other errors:

```
try:
    print (x)
except NameError:
    print("Variable x is not defined")
except:
    print("something else went wrong")
```

output: Variable x is not defined

Else :

You can use the `else` keyword to define a block of code to be executed if no errors were raised:

Example :

In this example, the try block does not generate any error:

```
try:
    print("hello")
except:
    print("something went wrong")
else:
    print("nothing went wrong")
```

output : hello

nothing went wrong

Raise an Exception

As a python developer you can choose to throw an exception if a condition occurs.

To throw (or raise) an exception, use the `raise` keyword.

Example :

Raise an error and stop the program if x is lower than 0:

```
x=-1
if x<0:
    raise Exception("sorry,no numbers below zero")
```

output : Exception: sorry,no numbers below zero

- the raise keyword is used to raise an exception.
- You can define what kind of error to raise, and the text to print to the user.

Example :

Raise a TypeError if x is not an integer:

```
x="hello"
if not type(x) is int:
    raise TypeError("only integers are allowed")
```

output: TypeError: only integers are allowed

Code Challenge : try and except

Test your understanding of Python try...except by completing a small coding challenge.

Instructions

Inside the editor, complete the following steps:

1. Write a **try** block that tries to print a variable **x** (which is not defined)
2. Write an **except** block that prints **"An error occurred"**
3. Write a **finally** block that prints **"Execution complete"**

```
try:
    print (x)
except:
    print("an error occurred")
finally:
    print("Execution complete")
```

output : an error occurred

Execution complete